系统 6:HyperLogLog

用少量空间估计不同元素的数量

A 问题是什么

一份访问日志有很多行，但究竟有多少个不同用户？精确方法需要保存出现过的用户集合；HyperLogLog（HLL）只保存一组很小的统计量，估计去重后的数量。Basic 按给定公式实现固定桶数的 HLL，观察重复输入和估计误差；Advance 增加合并、紧凑存储、稀疏表示和参数实验，理解流式统计怎样在空间与精度之间取舍。

阶段	工作量	难度	主要成果
Basic	一周约 10-20 小时	medium	固定 16 个寄存器的 HLL，支持插入与估计
Advance	Basic 后追加六周	hard	支持合并、6 位存储及稀疏 / 稠密转换的 HLL

需要 C++17、数组、循环和简单浮点运算。公式直接给出，Basic 不要求概率论证明。easy 为按明确规则完成局部操作，medium 需要理解估计或位操作，hard 需要协调多种存储表示并验证一致性。哈希辅助函数已提供，不要求重新实现。

Basic：固定参数的 HLL

1 需要回答的问题

编号	问题	回答要求
Q1	数据条数和基数有什么区别?	对 `a,b,a,c,b` 分别给出两个数量；说明 HLL 估计哪一个
Q2	为什么先把元素转换成哈希值?	解释相同元素重复输入应得到相同值，以及分布不均会带来什么问题
Q3	桶号和 rank 分别是什么?	按本项目约定拆分哈希位，解释前导零与“第一个 1 的位置”
Q4	寄存器为什么取最大值，而不累加次数?	说明重复插入同一元素为什么不改变结果；HLL 不能用于枚举元素或精确成员查询
Q5	一个寄存器依次收到 rank `2,1,4,4`，最终保存什么? 空 HLL 应估计多少?	写出逐步变化，并用小基数修正解释空输入得到 0

2 代码任务与公式

用 16 个 `uint8_t` 寄存器 M，初始全为 0。固定 `p=4`、`m=2^p=16`。输入由 n 个非空字符串组成，建议 `0 <= n <= 100000`；`set<string>` 仅用于验证真实基数，不参与 HLL 的状态更新。

直接使用目录内 helpers/hash.hpp，其中包含 `hash64`、`bucketIndex`、`rho` 和 `hllAlpha`。该文件来自课程仓库，学生负责下面的 add 与 estimate 逻辑。

1. `h = hash64(s)` 得到 64 位哈希值。
2. 高 **p** 位是桶号 j，剩余低 `64-p` 位用于计算 rank；不能将桶号位再次计入 rank。
3. rank 是剩余位从高到低扫描时第一个 1 的位置，从 1 开始。剩余位全为 0 时为 `64-p+1`，Basic 中最大为 61。可直接调用提供的 `rho`。
4. 更新 `M[j] = max(M[j], rank)`。

估计时先计算：

```
Z = sum(2^(-M[j]))
E = alpha_m * m * m / Z
V = 值为 0 的寄存器数量
若 E <= 2.5*m 且 V > 0，则 E = m * ln(m / V)
```

Basic 中 `alpha_m=0.673；ln` 用 C++ 的 `std::log。m / V` 必须使用浮点除法。`2^(-M[j])` 用 `std::ldexp(1.0, -int(M[j]))` 或 `std::pow` 计算，不能使用 C++ 的按位异或 `^`。估计值以浮点数输出即可。

空输入时 V=m，修正结果为 0。估计不必恰好是整数，也不保证对每个数据集都接近真实值。Basic 的 16 个桶主要用于理解流程，不追求低误差。

3 测试与验收

检查空输入估计为 0；同一个字符串插入一次与插入一百次，寄存器和估计必须完全相同。测试寄存器更新 `2→2→4→4`，并核对桶号与 rank 的边界。

再准备不同真实基数的数据，如 10、100、1000，每组加入重复值，输出真实基数、估计值和绝对相对误差 `abs(E-n)/n`；n=0 时误差显示“不适用”。保存生成规则，不能把某次估计碰巧很准当作所有数据上的保证。

提交源码、编译命令、测试记录和 Q1-Q5。Basic 以更新规则、给定公式和不变量验收，不设置“任意输入误差必须小于 1%”的错误要求，也不要求 presentation。若课程设置 OJ，宜测寄存器状态及公式结果的数值容差，而非每组数据的估计精度。

4 参考资料

先读 B1 的基数与分桶，再结合本 README 写程序；B2、B3 补充应用和哈希，B4 作英文对照。中文文章中的符号或位划分可能不同，代码统一采用上面的规则。

编号	资料与语言	建议范围
B1	杜沁园：探索 HyperLogLog 算法，中文	基数、分桶、调和平均的直观介绍；跳过 Java 实现和推导
B2	Apache Doris：HLL 近似去重，中文	使用场景与近似计数，数据库产品的精度描述不等于本项目固定 16 桶的精度
B3	Hello 算法：哈希算法，中文	哈希分布与碰撞，不要求设计哈希函数
B4	Redis：HyperLogLog，英文	PFADD、PFCOUNT 的用途，不要求部署 Redis

Advance：合并与存储表示

1 Goals 与难度

G1-G7 均需完成，G6 选择一个阅读方向。

Goal	完成要求	难度
G1	支持 `4 <= p <= 14`，实现兼容 HLL 的逐桶合并	easy
G2	实现每寄存器 6 位的 DenseHLL，读写结果与普通数组一致	medium
G3	实现仅保存非零桶的 SparseHLL，并在指定阈值转为 DenseHLL	hard
G4	实现明确格式的序列化与反序列化，恢复后仍可更新与合并	medium
G5	对照精确集合，比较不同 p 的误差与不同表示的空间	medium
G6	阅读一个相关方向，解释统计或工程思想	见方向表
G7	整理源码、可复现实验、阅读笔记和 presentation	medium

2 参数、估计与合并

参数改变后 m=`2^p`，继续使用 Basic 的位划分和小基数修正。`alpha_m` 使用提供的 `hllAlpha`：m 为 16、32、64 时分别为 0.673、0.697、0.709，其他取 `0.7213/(1+1.079/m)`。教学测试规模至多约一百万个不同元素，采用给定的 64 位哈希，不要求实现更大范围的偏差修正或兼容 Redis 的编码。

合并要求 p、哈希算法、哈希参数及位划分完全一致，不兼容时拒绝。逐桶执行 `M[i]=max(M[i], other.M[i])`，不能把两个估计值直接相加。对有交集的 A、B，验证“分别构建后合并”的所有寄存器，与“把 A∪B 的数据直接加入”的结果一致。稀疏与稠密表示也应能相互合并。

3 存储怎么实现

表示	要求
普通数组	复用 Basic，作为所有状态的验证基线
DenseHLL	用字节数组紧凑存放 m 个 6 位寄存器，长度为 `(6*m+7)/8` 字节；实现 `get(i)`、`set(i,v)`，覆盖跨字节边界
SparseHLL	用 `map< 桶号,rank>` 只保存非零值；当非零桶数大于 `m/4` 时转换为 DenseHLL，此后不再转回

6 位能表示 0-63，本项目所有合法 rank 都在范围内。可按位逐个读写实现 DenseHLL，不要求一次用复杂位运算完成。统一逻辑接口可用普通函数，不要求复杂的面向对象设计。

稀疏阈值是便于验收的教学规则，不能据此认定 `map` 在该阈值之前一定比位数组更省内存。报告稀疏条目数、序列化字节数与已知容器开销，区分逻辑存储与运行时实际占用。

4 序列化与实验

序列化为内存中的字节序列即可，格式自行规定并写进运行说明：至少包含格式版本、p、表示类型、哈希方案标识和数据长度。整数采用明确字节序；稀疏条目按桶号排序；反序列化检查长度、p、桶号和 rank 范围。无需真实文件读写、网络传输或缓存估计值。

验证空状态、稀疏状态、刚过阈值的转换、跨字节寄存器、合并与序列化往返。普通数组、Dense、Sparse 在相同输入下的逻辑寄存器须完全一致，估计也应在浮点容差内一致。

G5 至少比较 p=4、8、12 与三个真实基数。每个组合用至少十组不同元素集合重复试验，保留固定的数据生成种子；仅打乱同一集合的顺序不会构成新的独立试验。报告平均绝对相对误差和波动范围，再说明增加寄存器的代价。理论中常见的标准误差约 `1.04/sqrt(m)` 是统计尺度，不是每次估计误差的上界。HLL 原论文

5 参考资料与可选方向

编号	资料与语言	建议范围
A1	探索 HyperLogLog 算法，中文	分桶、估计、小基数修正、合并；以给定公式核对变量与条件
A2	Apache Doris：HLL 近似去重，中文	多批数据的聚合及应用，不要求运行 SQL
A3	Flajolet 等：HyperLogLog 原论文，英文	摘要、算法图与误差结论，不要求阅读证明
A4	Google：HyperLogLog in Practice，英文	摘要和工程改进动机，按兴趣读稀疏表示与偏差修正
A5	Redis：HyperLogLog，英文	观察实际系统如何提供估计与合并接口

G6 可选方向	难度	需要回答的问题
分布式统计与合并（A1、A2、A5）	medium	为什么可以先各自统计再合并？为什么重复合并同一状态不应增加结果？
概率估计与工程修正（A3、A4）	hard	原始估计在小数据时为何需要修正？更好的统计精度与存储设计分别改变了什么？

选读仅要求解释思想。最终提交源码、运行说明、格式说明、往返与合并测试、误差 / 空间表、简短笔记与 presentation。

B 工作量安排与 Checkpoint

Basic：一周

内容	预计用时
理解基数、分桶与 rank	4-5 小时
使用辅助函数并实现更新、估计	3-5 小时
精确基线、重复输入及公式测试	3-4 小时
Q1-Q5 与整理	2-3 小时
合计	12-17 小时，预留至 20 小时

Checkpoint：重复输入不改变状态，空输入为 0，能解释每个公式变量与估计误差。

Advance：追加六周

时间	对应 **Goal**	**Checkpoint**
第 1-2 周	G1：easy；G2：medium	参数化与合并正确，6 位寄存器与普通数组逐项一致
第 3-4 周	G3：hard；G4：medium	稀疏转换与序列化往返保持状态，拒绝不兼容合并
第 5-6 周	G5、G7：medium；G6：按方向	完成重复实验、空间比较、阅读与展示

C 问题的来龙去脉与发展过程

数据流和大规模日志让“保存全部不同元素”变得昂贵。概率计数和 LogLog 等方法通过观察哈希位的统计特征估计集合大小；2007 年的 HyperLogLog 进一步改进多个桶的统计汇总方式，在少量空间下获得较好的误差表现。HyperLogLog 原论文

后续工程工作继续关注小基数偏差、稀疏表示和存储效率；数据库与缓存系统又把它包装成便于调用的聚合接口。该项目从固定寄存器走到合并与表示转换，学习“估计规则”与“如何保存估计状态”这两个层次。HyperLogLog in Practice、Apache Doris HLL